Implementing ResNet and DenseNet for Face Recognition on LFW (Labeled Faces in the Wild) dataset

Step 1

- Ensure you have the following Python packages installed: TensorFlow
- NumPy
- Pandas
- Matplotlib
- scikit-learn

- Code: **pip install tensorflow numpy pandas matplotlib scikit-learn**

Step 2: Data Loading

```
import os

import tensorflow as tf


from tensorflow.keras import layers, models
from tensorflow.keras.applications import ResNet50, DenseNet121
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
```

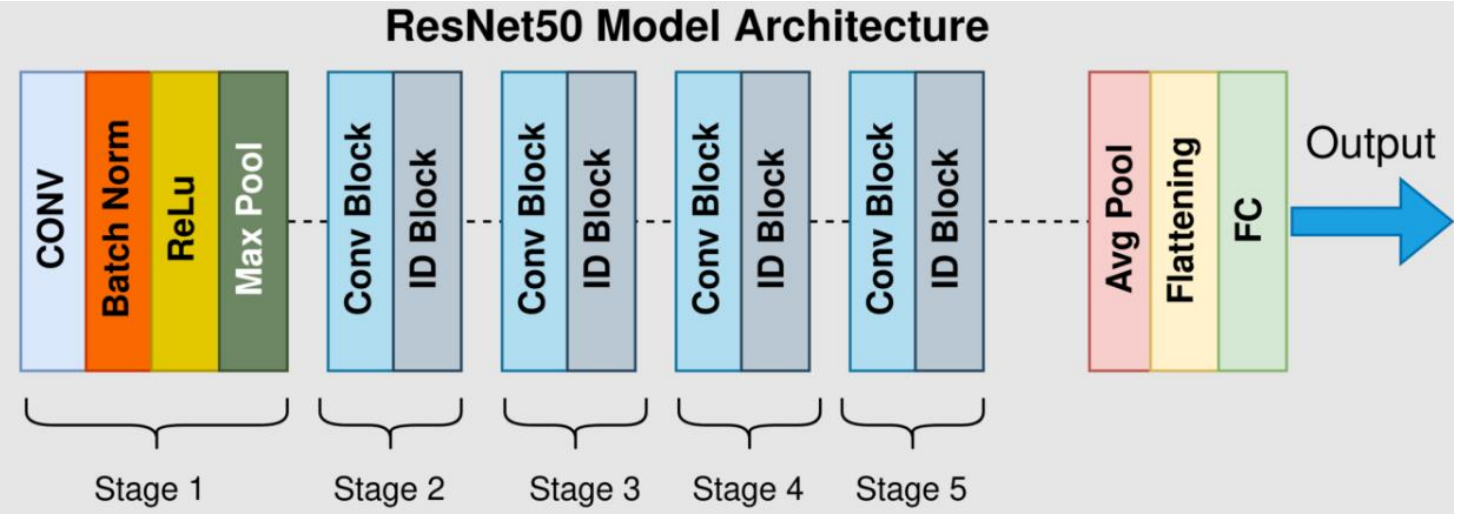
Step 3: Image preprocessing

- Resize all images to **224×224 pixels** for compatibility with ResNet and DenseNet.
- Normalize pixel values to the range **[0, 1]**.
- Perform **data augmentation** (horizontal flipping, rotation) to improve generalization.

- **Dataset Splitting**:
  - Use **80% of data for training** and **20% for validation**.

Step 4: Build the model

- **Load Pretrained ResNet**
  - Use **ResNet50** from TensorFlow Keras applications.
  - Load **pre-trained ImageNet weights**.
- **Modify Final Layers**:
  - Replace the **top classification layer** with a custom pooling + dense layer.
  - Set the final dense layer's **units equal to the number of classes** (identities).



Load Pretrained ResNet

```
def build_resnet(num_classes):
    base_model = ResNet50(
        weights="imagenet",
        include_top=False,
        input_shape=(224, 224, 3)
    )

    base_model.trainable = False  # Transfer learning
```

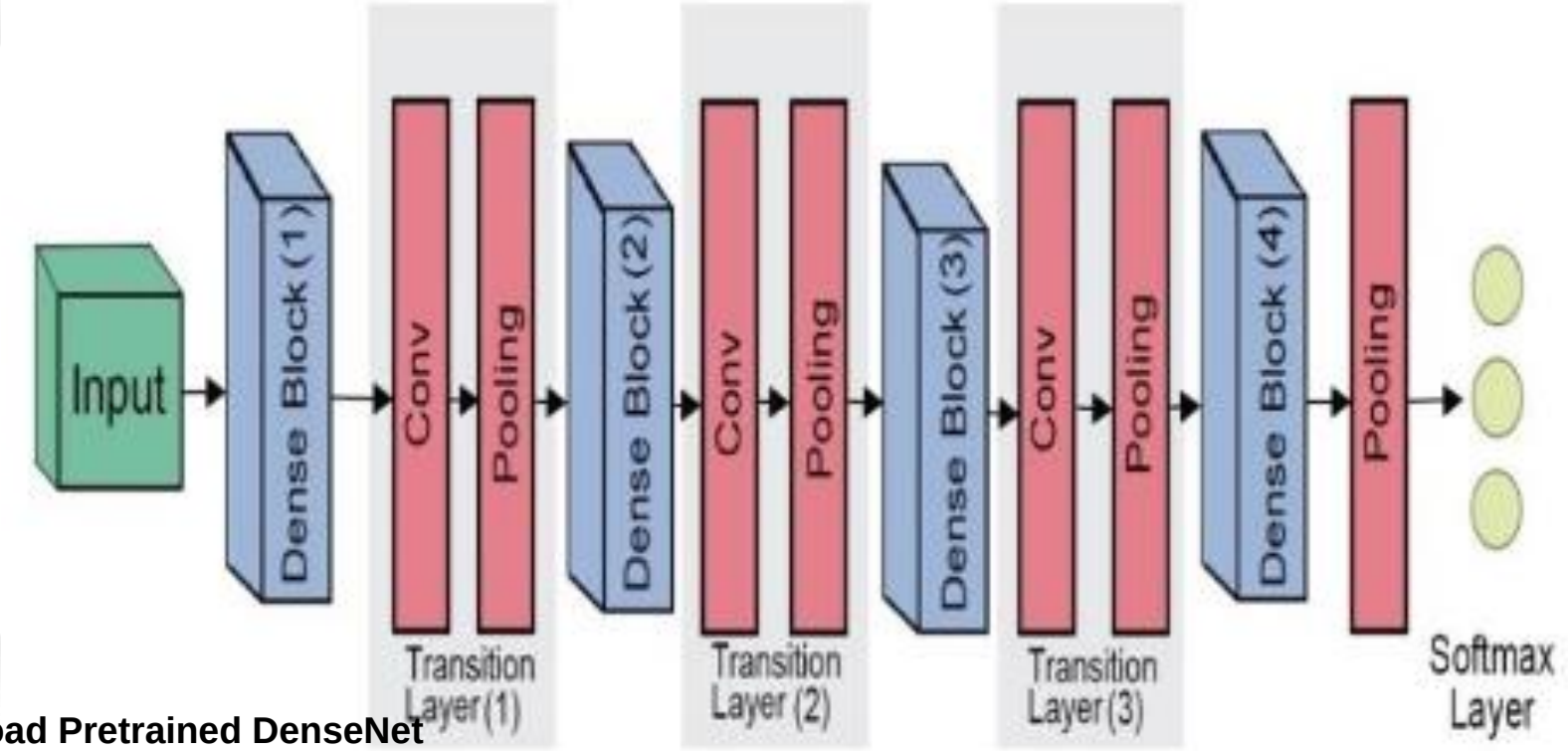
Step 5: Build DenseNet Model

Use DenseNet121:

- Similar to ResNet, it uses **pre-trained ImageNet weights**.
- Efficient because it **passes feature maps across layers**.

Modify Final Layers:

- Replace the **default classification layer** with a custom layer.
- Use **Softmax activation** for classification.



Load Pretrained DenseNet

```
def build_densenet(num_classes):
    base_model = DenseNet121(
        weights="imagenet",
        include_top=False,
        input_shape=(224, 224, 3)
    )

     base  model.trainable = False
```

Step 6: Train both the models

- **Train both models** with **early stopping** to prevent overfitting.

- Train ResNet model:
  - Use training data
  - Validate on validation data
  - Save best model
- Train DenseNet model:
  - Same procedure as ResNet

```
callbacks_resnet = [
    EarlyStopping(patience=5, restore_best_weights=True),
    ModelCheckpoint("resnet_lfw.h5", save_best_only=True)
]

callbacks_densenet = [
    EarlyStopping(patience=5, restore_best_weights=True),
    ModelCheckpoint("densenet_lfw.h5", save_best_only=True)
]


Add callbacks model.fit
history_resnet = resnet_model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=20,
    batch_size=32,
    callbacks=callbacks_resnet
)
```

Step 7: Evaluate and Compare Models

- Load best trained models
- Evaluate ResNet:
  - Get accuracy and loss
- Evaluate DenseNet:
  - Get accuracy and loss
- C ompare both models
- Generate training and validation Accuracy for both